

Diffusion-ES: 面向自动驾驶与零样本指令遵循的免梯度扩散规划

Brian Yang Huangyuan Su Nikolaos Gkanatsios Tsung-Wei Ke
Ayush Jain Jeff Schneider Katerina Fragkiadaki
Carnegie Mellon University

摘要

扩散模型擅长为决策与控制建模复杂且多模态的轨迹分布。近期提出的奖励梯度引导去噪方法，旨在生成既能最大化可微奖励函数、又能保持扩散模型所捕获数据分布下似然度的轨迹。奖励梯度引导去噪要求奖励函数可微，且需同时拟合干净样本与加噪样本，这限制了其作为通用轨迹优化器的适用性。本文提出 Diffusion-ES，该方法将免梯度优化与轨迹去噪相结合，在保持数据流形的同时优化黑盒不可微目标。Diffusion-ES 在进化搜索过程中从扩散模型采样轨迹，并使用黑盒奖励函数进行评分。它利用截断扩散过程对高分轨迹进行变异，该过程仅施加少量加噪与去噪步骤，从而大幅提升解空间的探索效率。实验表明，Diffusion-ES 在 nuPlan（一个成熟的自动驾驶闭环规划基准）上达到了最先进的性能。Diffusion-ES 优于现有的基于采样的规划器、反应式确定性或基于扩散的策略以及奖励梯度引导方法。此外，与先前的引导方法不同，本文方法能够优化由少样本大语言模型提示生成的不可微语言形式奖励函数。在人类教师发出遵循指令的引导下，本文方法能够生成新颖且高度复杂的行为，例如激进的蛇形变道，这些行为在训练数据中并不存在。这使得我们能够解决 nuPlan 中最困难的场景，这些场景超出了现有轨迹优化方法和驾驶策略的能力范围。¹

1. 引言

扩散模型在决策与控制领域已展现出对高度复杂且多模态轨迹分布建模的卓越能力 [1, 26]。奖励梯度引导 [26, 34, 62] 通过在去噪扩散步骤与将奖励梯度反向传播至加噪轨迹之间交替进行，实现了可微奖励函数的测试时优化。通过这种方式，采样得到的轨迹被推向轨迹数据流形，同时最大化当前奖励函数 [26]。这种奖励函数与轨迹扩散的解耦，使得单一轨迹扩散模型能够在测试时用于最大化多种奖励函数。奖励梯度引导要求奖励函数可微，并且需要在含噪与干净轨迹上均进行拟合，这通常需要重新训练。这限制了其作为通用轨迹优化求解器的适用性。本文提出扩散进化搜索（Diffusion-ES），一种用于优化不可微黑盒目标的奖励引导去噪方法，该方法利用扩散模型采样并变异轨迹，并由仅作用于干净、最终去噪样本的奖励函数进行引导。将扩散与基于采样的优化方法简单结合并不奏效：诸如交叉熵方法（CEM）[50] 或模型预测路径积分（MPPI）[66] 等基于采样的优化器，通常需要跨多轮选择与变异的大量样本群体才能收敛到良好解，而结合去噪推理的计算成本后，会导致搜索过程极其缓慢。在扩散进化搜索中，高分轨迹通过截断的扩散-去噪过程进行变异，即添加少量噪声并将其去噪还原，如图 2 右所示。所添加的噪声量在搜索迭代过程中逐步减少，使得扩散进化搜索在计算上可行。扩散进化搜索中用于测试时优化的轨迹扩散模型，原则上可以以任何场景相关信息为条件，将采样范围缩小到与场景相关的轨迹分布。事实上，条件信息的数量控制着一个连续谱系，介于

¹ 项目主页: diffusion-es.github.io

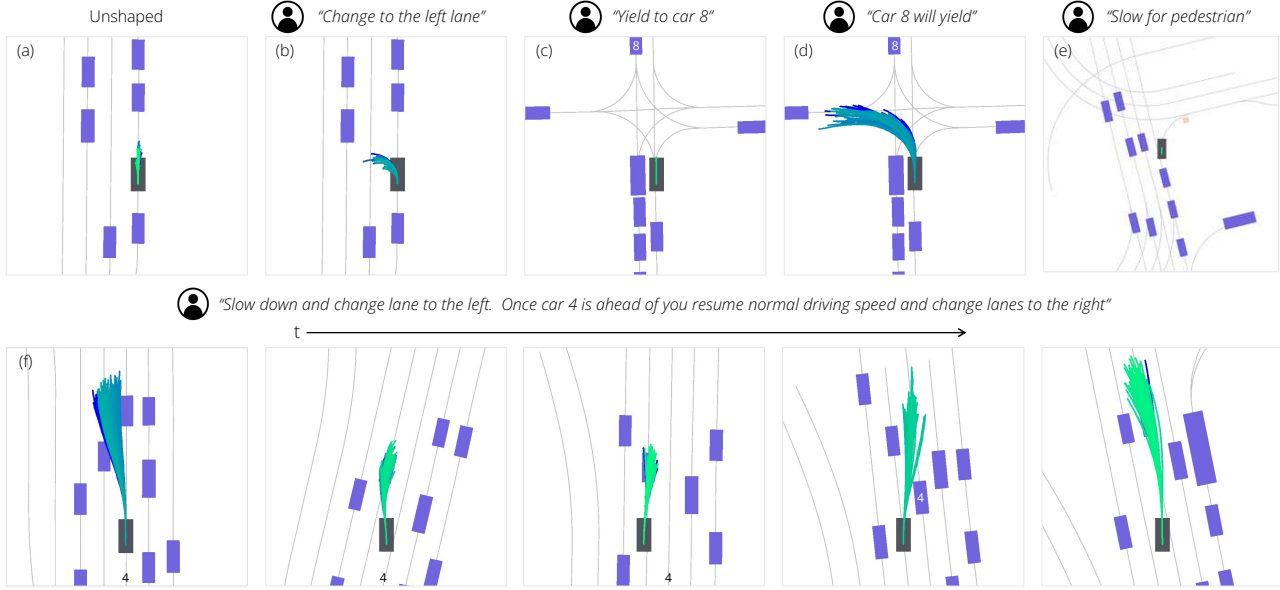


图 1. 扩散-ES 是一种针对任意奖励函数的测试时轨迹优化方法，结合了生成式轨迹模型与基于采样的搜索。(a) 通过优化通用驾驶奖励函数（鼓励沿车道行驶、避免碰撞并遵守交通标志）生成的轨迹。扩散-ES 在 nuPlan 中达到了最先进的驾驶性能 [5]。(b-f) 扩散-ES 通过测试时优化语言塑造的奖励函数，遵循自然语言的驾驶指令，无需任何额外训练。我们提示大语言模型将语言指令映射为塑造驾驶奖励函数的程序，然后用扩散-ES 进行优化。

先训练后测试的学习与测试时规划，以及推理速度与分布外泛化之间的相应权衡：**1.** 条件信息越多，从中抽取轨迹样本的分布越窄，搜索越快。在极端情况下，不使用测试时奖励优化，我们的扩散模型在测试时作为反应式策略运行。实际上，许多方法将扩散策略 [9] 训练为条件扩散轨迹预测模型，使用奖励作为轨迹扩散模型的条件信息 [11]，或通过强化学习 [36, 63] 或模仿学习 [41] 微调扩散策略。虽然这些方法可以处理黑盒奖励函数或需要模仿的良好行为，但我们表明，直接使用它们或在测试时使用奖励引导，往往不如对无条件扩散模型进行奖励引导去噪，后者完全解耦了轨迹建模与奖励建模。**2.** 条件信息越少，从中抽取轨迹样本的分布越宽，搜索越慢，但对分布外任务和场景的泛化越好，这些任务和场景需要训练数据中不存在的新颖轨迹与场景上下文配对。实际上，这是测试时规划优于先训练后测试学习的前提：对能量函数组合 [14, 18] 进行测试时优化，这里指轨迹数据分布的能量与任意奖励函数的能量，应该能够合成训练时未见的新颖行为。

我们展示了扩散进化策略（Diffusion-ES）在与轨迹上的无条件扩散模型结合时，如何纯粹通过扩散引导的黑盒奖励最大化实现最先进的规划性能。我们的方法在 nuPlan [5] 上进行了评估，这是一个建立在真实驾驶日志和估计的真值感知之上的成熟驾驶基准。我们在闭环驾驶中达到了最先进的性能，与之前的 SOTA 方法 PDM-Closed（一种针对 nuPlan 基准定制的基于采样的规划器）[12] 以及反应式驾驶策略 [46, 53]（确定性的或基于扩散的）的性能相当。此外，我们通过测试时优化使用少样本大语言模型提示生成的语言形状奖励函数，展示了扩散进化策略的灵活性。利用语言指令，我们可以解决最具挑战性的 nuPlan 场景，并合成全新的驾驶行为。然后，我们测试了我们的模型和基线在优化生成的奖励函数以引出期望行为方面的能力。通过我们的方法进行指令遵循所生成行为的定性示例见图 1。我们表明扩散进化策略显著优于 PDM-Closed、其他基于采样的规划器，以及扩散进化策略的消融版本——这些版本要么将扩散模型条件化于周围场景，要么完全不使用任何引导。

总之，我们的贡献如下：

1. 我们提出了扩散进化策略（Diffusion-ES），一种用于优化黑盒目标的轨迹优化方法，该方法使用

一种轨迹扩散模型，用于在基于采样的搜索过程中采样和变异轨迹提案。我们证明扩散进化搜索（Diffusion-ES）在 nuPlan 的闭环驾驶中达到了工程化规划器的最先进性能，并且在优化需要灵活驾驶行为（超越车道跟随）的更复杂奖励函数时，其性能远超这些规划器。据我们所知，这是首次将进化搜索与扩散模型相结合的工作。2. 我们证明扩散进化搜索可用于遵循语言指令，并通过优化大语言模型（LLM）形式的奖励函数来引导自动驾驶车辆的闭环驾驶行为，且无需任何语言与动作的训练数据。我们展示了这种指令遵循能够解决 nuPlan 中最具挑战性的驾驶场景。3. 我们对模型进行了广泛的消融研究，通过改变条件信息的数量，清晰地揭示了驾驶中推理速度与分布外（OOD）泛化之间的权衡。我们相信扩散进化搜索将作为一个通用的轨迹优化器对社区有用，其应用范围超越驾驶领域。我们的代码和模型将在发表后公开，以帮助复现，项目网页为：diffusion-es.github.io。

2. 相关工作

用于决策与轨迹优化的扩散模型 扩散模型 [21, 40, 57, 58] 通过迭代去噪过程学习逼近数据分布，并在图像生成上取得了令人瞩目的成果 [13, 45, 49, 51]。它们已被用于操作任务的模仿学习 [9, 41, 47, 67]、可控车辆运动生成 [6, 27, 74] 以及操作任务的视频生成 [15, 70]。[28, 75] 的工作使用扩散模型预测离线车辆轨迹。据我们所知，这是首次在闭环驾驶中使用扩散模型的工作。自动驾驶中的学习与规划 通过模仿驾驶示范来学习驾驶在自动驾驶汽车的研究与开发中十分普遍 [2, 3, 7, 8, 10, 32, 43, 44, 46, 73]。许多杰出的模仿方法假设底层动作分布是单峰的，这在从多模态专家示范中训练时会产生问题。已提出能够更好地处理多模态轨迹预测的目标与架构 [11, 17, 31, 39, 56, 61]。我们表明扩散模型非常适合驾驶，并可用于从多模态示范中合成丰富复杂的行为。另一方面，传统自动驾驶系统在决策上完全不依赖学习，而是依赖在线优化人工设计的代价函数。

[16, 38, 77]。近期，PDM-Closed [12] 在 nuPlan 驾驶基准上达到了最先进的性能，其完全依赖于测试时规划与启发式方法选择轨迹提案。其他先前工作旨在通过基于学习代价地图的采样规划 [72] 或基于学习动力学模型的梯度优化 [48]，将离线学习的优势融入测试时规划。本文通过展示基于扩散的生成模型如何与采样规划相结合，扩展了这一研究方向。自动驾驶的语言条件策略 近期，面向驾驶的语言条件策略取得了显著进展。GAIA-1 [22] 是一种生成式世界模型，能够利用视频、语言和动作合成符合给定语言指令的驾驶场景，实现多模态视频生成。然而，GAIA-1 并不执行任何实际控制输入。基于互联网规模文本训练的大语言模型在适当提示下，对多种下游语言任务展现出令人印象深刻的零样本推理能力，且无需任何权重微调 [4, 35, 64]。近期研究表明，通过适当的计划式或程序式提示，大语言模型可将语言指令映射为语言子目标 [23, 24, 68, 76]、动作程序 [19, 33, 60] 或代价地图 [25]。本文采用大语言模型的少样本提示来塑造驾驶奖励函数。我们扩展了先前方法，利用 Python 生成器产生在多次调用间保持内部状态的奖励函数。[37, 65, 69] 的工作使用大语言模型根据高层场景描述和语言指令预测底层控制信号。然而，这些方法均未在闭环驾驶上评估性能，而闭环驾驶比开环轨迹预测 [12] 困难得多。[55] 与本文相似，但仅考虑了高度简化的驾驶设置，且未在具有强基线的标准化基准上报告结果。

3. 方法

3.1. 背景

扩散模型 扩散模型通过反转前向扩散过程来捕捉概率分布 $p(x)$ ，该前向过程逐步向初始样本的中间分布 x 添加高斯噪声。所添加的噪声量取决于预定义的方差调度 $t=1$ ，其中 T 表示扩散时间步的总数。在扩散时间步 t ，前向

$\beta_t \in (0, 1)$ 扩散过程使用公式向 x 添加噪声，

其中 $\epsilon \sim \mathcal{N}(0, 1)$ 是来自具有相同

ward diffusion process a 维数的高斯分布的样本

$$x_t = \sqrt{\bar{\alpha}_t}x + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

from a Gaussian distribution with the same dimensionality as x . Here, $\alpha_t = 1 - \beta_t$, and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. For denoising过程中，神经网络 $\hat{\epsilon} = \epsilon_\theta(x_t; t)$ 以带噪声的

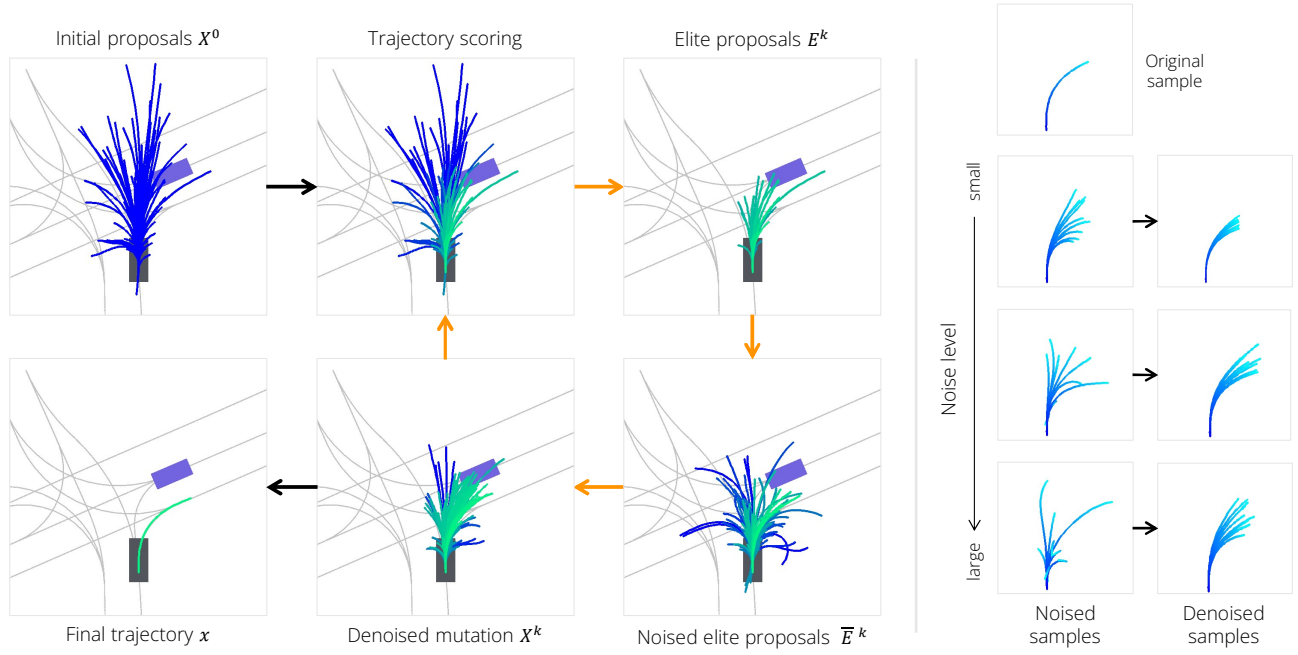


图 2. Diffusion-ES 概述。左：使用 Diffusion-ES 的生成式进化搜索。轨迹按奖励着色（蓝色为低，绿色为高）。右：我们可视化不同噪声水平下的变异。颜色表示沿轨迹的时间步。虽然仅靠噪声扰动可能导致不切实际的轨迹，但去噪有助于将样本投影回轨迹数据流形。

样本 x_t 和扩散时间步 t 作为输入，并学习预测添加的噪声 ϵ 。为了从学习到的分布 $p_\theta(x)$ 生成样本，我们首先从先验分布 $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 中抽取一个样本，并使用 ϵ_θ 对该样本迭代去噪 T 次。 ϵ_θ 的应用取决于指定的采样计划 [21, 58]，该计划以从 $p_\theta(x)$ 采样的 x_0 结束。扩散模型可以轻松扩展为对 $p(x|\mathbf{c})$ 进行建模，其中 $\mathbf{c}$ 是某种条件信号，例如预期的未来奖励，通过向去噪神经网络 ϵ_θ 添加额外输入来实现。进化策略 进化策略 (ES) 是一类基于种群的免梯度优化算法，可以在没有任何训练的情况下最大化任意黑盒奖励函数 $R(x) : \mathbb{R}^d \rightarrow \mathbb{R}$ ，其中 x 是我们优化的变量。ES 迭代更新搜索分布 $q(x)$ 以最大化期望奖励 $\mathbb{E}_{x \sim p_\theta(x)}[R(x)]$ 。虽然基于分布的 ES 方法（如 **CEM** 和 **CMA-ES**）显式地表示 $q(x)$ （通常为单峰高斯分布），但我们也可以将 $q(x)$ 非参数化地表示为一组高性能解，而无需对 q 的函数形式做出强假设。

3.2. Diffusion-ES

Diffusion-ES 是一种轨迹优化方法，利用免梯度进化搜索从训练好的扩散模型中进行奖励引导采样，适用于任意黑盒奖励函数 $R(x)$ 。具体来说，我们使用

训练好的扩散模型 ϵ_θ 来初始化样本种群，并使用截断的扩散-去噪过程在保持数据流形的同时对样本进行变异。Diffusion-ES 的控制流程如图 2（左）和算法 1 所示。使用扩散采样初始化种群 我们首先使用扩散模型采样一个包含 M 个轨迹样本的初始种群 X^0 ：

$$X^0 = \{x_i\}_{i=1}^M \sim p_\theta(x), \quad (1)$$

其中 X^k 为第 k 次迭代时的种群。这涉及对逆向扩散过程的完整遍历。鉴于我们使用的是无条件扩散模型，这些样本与场景无关，因此无需在每个时间步重新采样即可始终使用。我们还可以通过纳入其他方法生成的样本或混合上一时间步的解来修改初始种群，从而对优化过程进行热启动。样本评分 在每次迭代 k 中，我们对种群 $\{R(x_i)|x_i \in X^k\}_{i=1}^M$ 中的样本进行评分 $i=1$ 。需要注意的是，我们的种群由“干净”样本组成，因此无需能够处理“含噪”样本的奖励函数，这相较于执行基于分类器引导的引导方法，为我们提供了显著的灵活性。选择 我们利用奖励来决定应选择哪些样本传播至下一次迭代。与

MPPI[66] 类似，我们按如下方式重新采样 X^{k+1} ：

$$q(x) = \frac{\exp(\tau R(x))}{\sum_{i=1}^M \exp(\tau R(x_i))} \quad (2)$$

$$E^{k+1} = \{x_i \stackrel{\text{iid}}{\sim} q(x)\}_{i=1}^M, \quad (3)$$

其中 E^{k+1} 表示从第 k 次迭代中保留的精英集， τ 是可调的温度参数，用于控制 q 的锐度。使用截断扩散去噪进行变异 我们对 E^{k+1} 施加随机变异以进行探索。先前的进化搜索方法采用朴素的 Gaussian 扰动，未能利用关于数据流形的任何先验知识。我们的核心洞见在于利用截断的扩散-去噪过程对轨迹进行变异，使所得变异属于数据流形。我们可以运行前向扩散过程的前 t 步，以获得含噪精英样本 $\bar{E}^{k+1}$ ：

$$\bar{E}^{k+1} = \{\sqrt{\alpha_N}x + \sqrt{1 - \alpha_N}\epsilon | x \in E^{k+1}\}, \quad (4)$$

其中 $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{1})$ 。随后，我们可以运行逆向扩散过程的最后 t 步，再次对样本进行去噪，从而得到干净样本 X^{k+1} ：

$$X^{k+1} = \{x \sim p_\theta(x|\bar{x}) | \bar{x} \in \bar{E}^{k+1}\}. \quad (5)$$

在实践中，截断扩散过程的时间步数 t 是一个可调的时间相关超参数 t_k ，用于控制每次迭代 k 的变异强度。该过程如图 2（右）所示。我们发现，在 20 个搜索步骤内将变异扩散步数 t_k 从 5 线性衰减至 1，在我们的实验中效果最佳。

Algorithm 1 Diffusion-ES

```

1: Input: Diffusion model  $p_\theta$ , reward function  $R$ , search steps  $K$ , population size  $M$ , number of noising steps  $N$ , variance schedule  $\bar{\alpha} = \{\bar{\alpha}_i\}_{i=1}^N$ .
2: Initial proposals:  $X^0 \leftarrow \{x_i \sim p_\theta(x)\}_{i=1}^M$ 
3: for search step  $k \in (1, \dots, K)$  do
4:   Score proposals  $\{R(x_i) | x_i \in X^{k-1}\}_{i=1}^M$ 
5:   Compute distribution  $q(x) = \frac{\exp(\tau R(x))}{\sum_{i=1}^M \exp(\tau R(x_i))}$ 
6:   Sample elites from  $X^{k-1}$ :  $E^k \leftarrow \{x_i \stackrel{\text{iid}}{\sim} q(x)\}_{i=1}^M$ 
7:   Renoise elites  $\bar{E}^k \leftarrow \{\sqrt{\bar{\alpha}_N}x + \sqrt{1 - \bar{\alpha}_N}\epsilon | x \in E^k, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{1})\}$ 
8:   Denoise elites  $X^k \leftarrow \{x \sim p_\theta(x|\bar{x}) | \bar{x} \in \bar{E}^k\}$ 
9: end for
10: return output  $x = \arg \max_{x \in X^K} R(x)$ 

```

3.3. 利用大语言模型提示将语言指令映射为奖励函数

为遵循自然语言给出的驾驶指令，我们将其映射为黑盒奖励函数，并利用扩散进化策略（Diffusion-ES）进行优化。我们采用与

[29, 71] 该方法利用大语言模型（LLM）从语言指令中合成奖励函数。奖励函数具有可组合性，使我们能够将语言引导与其他约束无缝结合。这在驾驶场景中至关重要，因为我们需要同时优化多个不同目标（如安全性、驾驶员舒适度、路线遵循）。与先前工作类似，我们提供了一个 Python 应用程序编程接口（API），可用于提取道路场景中实体的信息。由于驾驶中许多基本奖励信号在不同场景间保持不变（例如碰撞避免或可行驶区域合规性），我们允许大语言模型编写奖励塑形（Reward Shaping）代码来修改基础奖励函数的行为，而非从头生成所有内容。奖励塑形可以添加辅助奖励项（例如密集的车道到达奖励）或重新加权现有奖励项。我们在图 3 中展示了生成的代码示例。我们的目标是处理具有时间依赖性的通用复杂语言指令，例如“向左变道，然后从右侧超车，然后驶出出口”。先前工作 [25, 71] 生成的是静态奖励函数，即在规划过程中保持固定。这使得仅通过奖励来表达顺序计划变得困难。我们发现，在代码中捕捉这些计划的一种更自然且简洁的方式是使用生成器函数，该函数在调用之间保留内部状态。我们所有的提示和代码示例均可在补充文件中找到。在第 4 节中，我们展示了优化这些语言塑形奖励函数如何合成符合语言指令的丰富且复杂的驾驶行为。

4. 实验

我们首先在 nuPlan [5] 上评估 Diffusion-ES 的闭环驾驶性能，nuPlan 是一个成熟的基准，利用估计感知来处理车辆、行人、车道和交通标志。我们的模型和基线在能够访问数据集提供的接近真实感知输出的条件下，评估其安全高效驾驶的能力。我们还考虑了一组驾驶指令遵循任务。我们通过大语言模型提示将指令映射为形状奖励函数，并评估 Diffusion-ES 和基线在优化生成的奖励函数以及准确遵循指令方面的能力。我们的实验旨在回答以下问题：1. 与现有的基于采样的规划器和用于轨迹优化的奖励梯度引导相比，Diffusion-ES 表现如何？2. 与直接将环境状态映射为车辆轨迹的最先进反应式驾驶策略相比，Diffusion-ES 表现如何？3. 假设能够访问给出指令的人类教师，能否解决最困难的 nuPlan 驾驶场景？

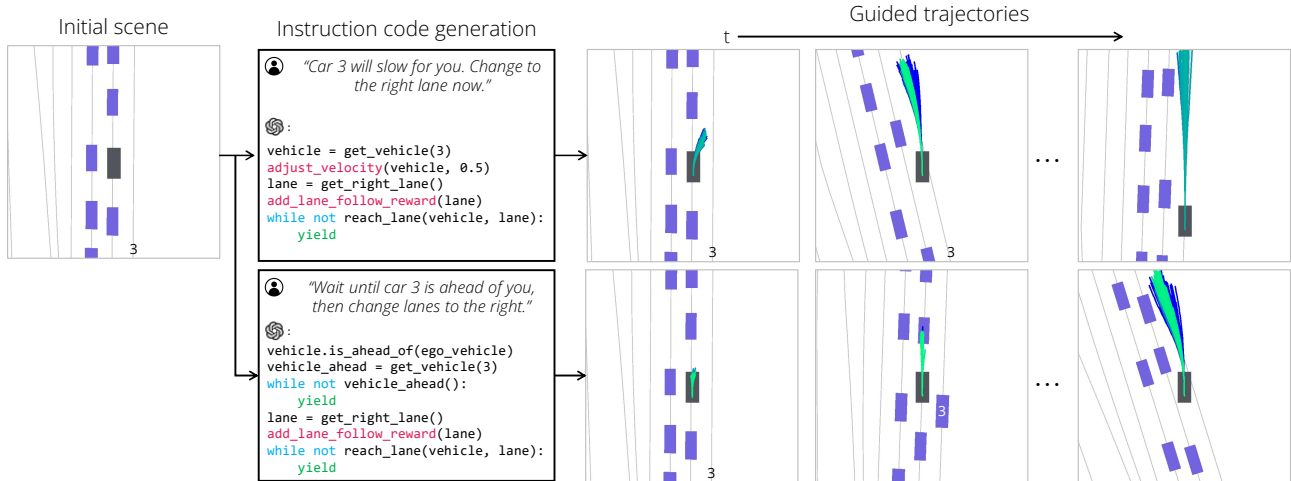


图 3. 利用大语言模型提示，通过语言指令塑造奖励函数，并使用扩散进化策略进行优化。针对同一初始场景，我们展示了两不同的语言指令，以及它们生成的奖励塑形代码和由扩散进化策略生成的轨迹。

在自然语言条件下，且不借助任何额外训练数据？4. 场景条件化对扩散模型是否有利于扩散进化策略（Diffusion-ES）？

4.1. 闭环驾驶

我们在 nuPlan Val14 规划基准（benchmark）上评估我们的模型 [12]。我们特别考虑 nuPlan 基准中的反应式智能体赛道，因为它是 nuPlan 评估设置中最困难且最贴近现实的。

模型设置与超参数 对于扩散进化策略（Diffusion-ES），我们在由二维位姿 (x, y, θ) 组成的自车轨迹上训练扩散模型，该轨迹预测未来 8 秒、频率为 2 赫兹，因此总动作维度为 48。与先前工作 [26] 不同，我们仅对动作分布建模，而非对状态和动作联合建模。实验中种群规模设为 $M = 128$ 。扩散模型采用 $T = 100$ 步去噪训练。

奖励函数 我们采用 PDM-Closed[12] 中评分函数的修改版本作为奖励函数。为计算奖励，我们使用 LQR 跟踪器将预测轨迹转换为低级控制输入。这些控制输入被输入到运动学自行车模型 [42] 中，该模型传播自车动力学。我们遵循 [12]，通过假设恒定速度来预测其他智能体的运动。然后根据 nuPlan 基准评估度量对这些模拟推演进行评分。我们还添加辅助奖励项，以惩罚与领先智能体的接近程度并强制执行速度限制。注意，由于跟踪器和用于评估交通违规的非可微启发式方法，该奖励函数不可微。

此外，训练模型回归奖励具有挑战性，因为 nuPlan 数据集不包含严重交通违规的实例。

评估度量 我们使用驾驶分数报告结果，该分数聚合了与交通规则合规性、安全性、路线进展和乘客舒适度相关的多个规划度量。这是 nuPlan 中使用的标准评估度量。

基线 我们考虑以下基线：• UrbanDriverOL[52]，一种确定性变换器策略，通过行为克隆和增强进行训练。与 [52] 不同，nuPlan 实现不执行闭环训练。• PlanCNN[46]，一种确定性模仿策略，使用 CNN 骨干编码栅格化鸟瞰图。• IDM[20]：一种启发式基于规则的规划器，调整速度以保持与领先车辆的安全距离。它也用于控制 nuPlan 中智能体的行为。• PDM-Closed[12]：一种基于 MPC 的规划器，使用车道中心线生成路径提议，并以与我们类似的方式推演轨迹。PDM-Closed 不是迭代优化奖励，而是在一轮评分后直接执行表现最佳的提议。它是 nuPlan Val14 基准上当前最先进的方法。• 扩散策略：我们考虑的一种扩散模型，以场景特征为条件直接预测车辆轨迹。我们使用 Urban Driver[53] 的变换器特征骨干编码场景特征。我们使用模仿学习和增强进行训练。这类似于 Diffusion-ES 中的无条件轨迹模型，但增加了对场景特征的条件。

	Method	Driving Score ($\uparrow$)
Train-then-test	UrbanDriverOL [53]	65
	PlanCNN [46]	72
	Diffusion policy	50
Test-time optimize	IDM [50]	77
	PDM-Closed [12]	92
	Diffusion-ES (ours)	92

表 1. 在 [12] 的 val14 划分中的闭环驾驶结果。我们的模型与先前基于非学习的规划器的性能相当，并显著优于所有其他模型。

我们在表 1 中展示了定量结果。我们得出以下结论：1. 扩散进化策略（Diffusion-ES）与先前最先进的 **PDM-Closed** 性能相当，并大幅优于所有其他基线。**PDM-Closed** 是一种基于采样的规划器，依赖领域特定启发式来生成轨迹提议，而扩散进化策略从数据中学习这些提议。两者都使用相似的奖励函数和场景中智能体的动力学模型。2. 反应式神经策略与测试时规划器之间存在巨大的性能差距，这一点在近期工作 [12] 中也被指出。我们假设这是因为与其他控制基准相比，nuPlan 具有更丰富的观测空间，场景中密集分布着动态参与者，其中许多与自车无关。这可能使基于学习的方法难以泛化，从而激发了测试时优化的需求。3. 扩散进化策略显著优于扩散策略。定性来看，我们的扩散策略倾向于随机变道，这导致自车更快进入分布外场景。扩散进化策略利用生成建模的表达能力，同时使用测试时优化来提高泛化能力。

4.2. 语言指令遵循

nuPlan 驾驶基准的一个缺点是它鼓励高度保守的驾驶行为。例如，PDM-Closed[12] 在 nuPlan Val14 基准中保持最先进水平，但无法变道，因为其路径提议仅考虑自车当前所在的车道。然而，在当前 nuPlan 基准中，变道对于良好的驾驶性能并非必需。为了评估扩散进化策略和基线在优化任意奖励函数方面的能力，我们考虑了八个语言指令遵循任务，每个任务取自 nuPlan 基准中现有的驾驶日志。在每个任务中，语言指令要求自车执行特定的驾驶操作以解决具有挑战性的驾驶场景。在大多数场景中，将没有示例

在 nuPlan 中实现指令行为。例如，车道穿梭任务要求自车在密集的城市交通中激进地变换多条车道。任务描述、语言指令和提示见附录。我们使用 3.3 节所述方法，根据语言指令生成可执行的 Python 代码，该代码调整第 4.1 节的初始奖励函数，从而为每个场景生成语言塑造的奖励函数。我们的模型和基线将优化相同的语言塑造奖励函数。

评估度量 我们使用任务成功率评估模型和基线，该指标衡量智能体成功完成指定任务的频率。为增加任务难度，我们在每个回合的零星时间间隔内向其他车辆智能体的 IDM 参数添加噪声，以随机化其行为。所有报告的分数均为十个随机种子的平均值。

基线 我们与以下基线进行比较：

- PDM-Closed：通过使用我们的语言塑造奖励函数替代原始奖励函数，使其适应该设置。
- PDM-Closed-Multilane：PDM-Closed 的修改变体，考虑更广泛的横向偏移路径，允许变道。
- 条件扩散-ES：使用条件扩散模型而非无条件扩散模型的扩散-ES。

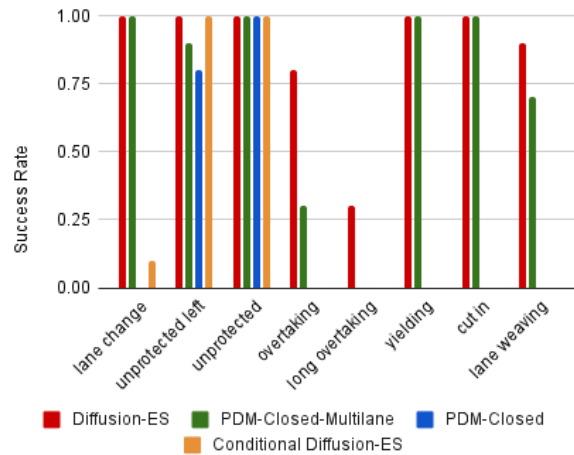


图 4. 遵循驾驶指令。扩散-ES 在优化复杂的语言塑造奖励函数方面优于所有基线。

图 4 展示了可控性任务的成功率。我们得出以下结论：1. 扩散-ES 优于所有基线。尽管 PDM-Closed-Multilane 的性能有显著提升

Method	Lane Error	Speed Error
CEM	2.34	2.05
MPPI	3.29	2.74
Reward-gradient guidance	1.22	0.96
Diffusion-ES (ours)	0.61	0.79

表 2. nuPlan 中的车道跟随。

与 PDM-Closed 相比，由于轨迹提议更加多样，但在 8 个任务中有 4 个任务仍弱于 Diffusion-ES。这凸显了依赖手工规则生成提议的弱点。2. 使用条件扩散模型时，Diffusion-ES 的性能显著下降。条件扩散模型更难引导，因为场景上下文导致符合分布的样本更少。

4.3. 车道跟随

为了将 Diffusion-ES 与奖励梯度引导进行比较，我们考虑一个简化的车道跟随任务，该任务具有可微的奖励函数，包含两项：一项惩罚偏离车道的横向偏差（车道误差），另一项惩罚偏离目标速度的偏差（速度误差）。我们从 nuPlan 中每种场景类型各采样一个，共 14 个场景，并报告所有场景的平均规划成本。

基线 我们考虑以下基线：• CEM [50]：一种广泛使用的进化策略方法，将搜索分布 q 参数化为高斯分布。CEM 在从 q 采样和将 q 拟合到最佳样本之间迭代。• MPPI [66]：与 CEM 类似，但 MPPI 不是保留固定数量的精英样本，而是按奖励比例采样。• 奖励梯度引导 [26]：我们在去噪过程中直接使用梯度下降优化真实规划目标。我们在表 2 中展示了定量结果。我们得出以下结论：1. 即使目标可微，Diffusion-ES 也优于可微奖励梯度引导基线。我们假设这是因为尽管真实奖励函数可用，但它可能无法为中间噪声轨迹提供合适的引导。这凸显了我们方法相对于先前工作的一个关键优势，即我们可以优化新目标，而无需在噪声样本上训练奖励回归器。2. 两种基于扩散的方法都显著优于未利用扩散的基于采样的规划器。这与我们的假设一致，即扩散引导比传统进化策略方法能更高效地优化轨迹。我们方法在所有实验设置中驾驶的视频可以在我们的项目页面 diffusion-es.github.io 上找到。

Method	Wallclock time (s)
Diffusion	1.11 ± 0.02
Diffusion-ES	5.85 ± 0.11
Diffusion-ES (<i>optimized</i>)	0.50 ± 0.01

表 3. Diffusion-ES 运行时间比较。

4.4. 运行时间分析

扩散演化策略（Diffusion-ES）经过少量优化即可实时运行。通过减少扩散步数 $T = 10$ 、缩小种群规模 $M = 32$ 并降低迭代次数 $K = 2$ ，本文方法能够以与模拟器相同的频率（2 赫兹）运行，仅以极小的性能代价（nuPlan 驾驶评分从 92 降至 91）。表 3 报告了 100 次试验中每个时间步推理的平均墙钟时间。

4.5. 讨论 - 局限性 - 未来工作

如第 4.4 节所示，本文方法确实引入了计算开销。本文认为，通过引入扩散建模的最新进展（如更快的采样器），这些问题可以得到缓解。本文的奖励函数假设其他智能体以恒定速度行驶，这显然有待改进。该假设还认为其他智能体无法对自行车做出反应，而这一点已被证明是自动驾驶规划器的主要局限性 [48]。然而，本文的指令遵循实验表明，即使无法实现完美预测，也能利用语言塑形奖励来解决最困难的驾驶场景。本文旨在未来工作中探索记忆提示的类比奖励塑形，以在无人类教师的情况下处理长尾场景。

5. 结论

本文提出了扩散演化策略（Diffusion-ES），一种用于黑盒奖励引导扩散采样的方法。实验表明，扩散演化策略能够在 nuPlan 中有效优化驾驶和指令遵循的奖励函数，并优于工程化采样规划器、反应式确定性或扩散策略以及可微奖励梯度引导。本文展示了如何在不使用任何语言-动作轨迹数据的情况下，仅通过大语言模型提示生成塑形奖励图，用于测试时优化，从而实现语言指令的遵循。未来工作将探索在无人类教师的情况下检索正确的奖励塑形，以处理长尾驾驶场景。实验揭示了扩散策略在场景条件化过程中推理速度与分布外泛化之间的权衡。未来工作将探索将此类搜索的结果摊销到快速反应式策略中，并平衡两个极端，以便根据场景灵活分配计算量。

致谢

本材料基于以下资助项目：美国国家科学基金会（NSF）职业奖、空军科学研究办公室（AFOSR）FA95502310257 奖、海军研究办公室（ONR）N000142312415 奖以及陆军研究实验室（ARL）第 48987.3.1130233 号奖。本材料中表达的任何观点、发现、结论或建议均为作者个人观点，并不一定反映美国陆军、国家科学基金会或美国空军的观点。

参考文献

- [1] Anurag Ajay, Yilun Du, Abhi Gupta, Joshua B. Tenenbaum, T. Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision-making? *ArXiv*, abs/2211.15657, 2022. 1, 2
- [2] Mayank Bansal, Alex Krizhevsky, and Abhijit S. Ogale. Chauffeurnet: Learning to drive by imitating the best and synthesizing the worst. *CoRR*, abs/1812.03079, 2018. 3
- [3] Mariusz Bojarski, Davide Del Testa, Daniel Dworakowski, Bernhard Firner, Beat Flepp, Prasoon Goyal, Lawrence D. Jackel, Mathew Monfort, Urs Muller, Jiakai Zhang, Xin Zhang, Jake Zhao, and Karol Zieba. End to end learning for self-driving cars, 2016. 3
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020. 3
- [5] Holger Caesar, Juraj Kabzan, Kok Seang Tan, Whye Kit Fong, Eric Wolff, Alex Lang, Luke Fletcher, Oscar Beijbom, and Sammy Omari. Nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles, 2022. 2, 5
- [6] João Carvalho, An T. Le, Mark Baierl, Dorothea Koert, and Jan Peters. Motion planning diffusion: Learning and planning of robot motions with diffusion models. *ArXiv*, abs/2308.01557, 2023. 3
- [7] Dian Chen and Philipp Krähenbühl. Learning from all vehicles, 2022. 3
- [8] Dian Chen, Brady Zhou, Vladlen Koltun, and Philipp Krähenbühl. Learning by cheating. *ArXiv*, abs/1912.12294, 2019. 3
- [9] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion, 2023. 2, 3
- [10] Felipe Codevilla, Matthias Müller, Alexey Dosovitskiy, Antonio M. López, and Vladlen Koltun. End-to-end driving via conditional imitation learning. *CoRR*, abs/1710.02410, 2017. 3
- [11] Alexander Cui, Abbas Sadat, Sergio Casas, Renjie Liao, and Raquel Urtasun. Lookout: Diverse multi-future prediction and planning for self-driving. *CoRR*, abs/2101.06547, 2021. 3
- [12] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning, 2023. 2, 3, 6, 7
- [13] Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis. *ArXiv*, abs/2105.05233, 2021. 3
- [14] Yilun Du, Conor Durkan, Robin Strudel, Joshua B. Tenenbaum, Sander Dieleman, Rob Fergus, Jascha Sohl-Dickstein, Arnaud Doucet, and Will Grathwohl. Reduce, reuse, recycle: Compositional generation with energy-based diffusion models and mcmc, 2023. 2
- [15] Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and P. Abbeel. Learning universal policies via text-guided video generation. *ArXiv*, abs/2302.00111, 2023. 3
- [16] Haoyang Fan, Fan Zhu, Changchun Liu, Liangliang Zhang, Li Zhuang, Dong Li, Weicheng Zhu, Jiangtao Hu, Hongye Li, and Qi Kong. Baidu apollo em motion planner. *arXiv preprint arXiv:1807.08048*, 2018. 3
- [17] Katerina Fragkiadaki, Jonathan Huang, Alex Alemi, Sudheendra Vijayanarasimhan, Susanna Ricco, and Rahul Sukthankar. Motion prediction under multi-modality with conditional stochastic networks, 2017. 3
- [18] Nikolaos Gkanatsios, Ayush Jain, Zhou Xian, Yunchu Zhang, Christopher Atkeson, and Katerina Fragkiadaki. Energy-based Models are Zero-Shot Planners for Compositional Scene Rearrangement. In *Robotics: Science and Systems*, 2023. 2
- [19] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14953–14962, 2023. 3
- [20] Dirk Helbing and Benno Tilch. Generalized force model of traffic dynamics. *Physical review E*, 58(1): 133, 1998. 6
- [21] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural*

Information Processing Systems, 33:6840–6851, 2020. 3, 4

- [22] Anthony Hu, Lloyd Russell, Hudson Yeo, Zak Murez, George Fedoseev, Alex Kendall, Jamie Shotton, and Gianluca Corrado. Gaia-1: A generative world model for autonomous driving. *arXiv preprint arXiv:2309.17080*, 2023. 3
- [23] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. *arXiv preprint arXiv:2201.07207*, 2022. 3
- [24] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models, 2022. 3
- [25] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. *arXiv preprint arXiv:2307.05973*, 2023. 3, 5
- [26] Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022. 1, 6, 8
- [27] Chiyu Jiang, Andre Cornman, Cheolho Park, Benjamin Sapp, Yin Zhou, and Dragomir Anguelov. Motion-diffuser: Controllable multi-agent motion prediction using diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9644–9653, 2023. 3
- [28] Chiyu “Max” Jiang, Andre Cornman, Cheolho Park, Benjamin Sapp, Yin Zhou, and Dragomir Anguelov. Motiondiffuser: Controllable multi-agent motion prediction using diffusion, 2023. 3
- [29] Pushkal Katara, Zhou Xian, and Katerina Fragkiadaki. Gen2sim: Scaling up robot learning in simulation with generative models, 2023. 5
- [30] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023. 13
- [31] Sanmin Kim, Hyeongseok Jeon, Jun Won Choi, and Dongsuk Kum. Diverse multiple trajectory prediction using a two-stage prediction network trained with lane loss. *IEEE Robotics and Automation Letters*, 8(4): 2038–2045, 2023. 3
- [32] Alex Kuefler, Jeremy Morton, Tim Allan Wheeler, and Mykel John Kochenderfer. Imitating driver behavior with generative adversarial networks. *CoRR*, abs/1701.06699, 2017. 3
- [33] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022. 3
- [34] Zhixuan Liang, Yao Mu, Mingyu Ding, Fei Ni, Masayoshi Tomizuka, and Ping Luo. Adaptdiffuser: Diffusion models as adaptive self-evolving planners. In *ICML*, 2023. 1
- [35] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *CoRR*, abs/2107.13586, 2021. 3
- [36] Cheng Lu, Huayu Chen, Jianfei Chen, Hang Su, Chongxuan Li, and Jun Zhu. Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning. *arXiv preprint arXiv:2304.12824*, 2023. 2
- [37] Jiageng Mao, Yuxi Qian, Hang Zhao, and Yue Wang. Gpt-driver: Learning to drive with gpt. *arXiv preprint arXiv:2310.01415*, 2023. 3
- [38] M Montemerlo, J Becker, S Bhat, and H Dahlkamp. The stanford entry in the urban challenge. *Journal of Field Robotics*, 7(9):468–492, 2008. 3
- [39] Nigamaa Nayakanti, Rami Al-Rfou, Aurick Zhou, Kratarth Goel, Khaled S. Refaat, and Benjamin Sapp. Wayformer: Motion forecasting via simple and efficient attention networks, 2022. 3
- [40] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171. PMLR, 2021. 3
- [41] Tim Pearce, Tabish Rashid, Anssi Kanervisto, Dave Bignell, Mingfei Sun, Raluca Georgescu, Sergio Valcarcel Macua, Shan Zheng Tan, Ida Momennejad, Katja Hofmann, and Sam Devlin. Imitating human behaviour with diffusion models, 2023. 2, 3
- [42] Philip Polack, Florent Althé, Brigitte d’Andréa Novel, and Arnaud de La Fortelle. The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles? In *2017 IEEE intelligent vehicles symposium (IV)*, pages 812–818. IEEE, 2017. 6
- [43] Dean A. Pomerleau. ALVINN: an autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems 1*, pages 305–313. San Francisco, CA: Morgan Kaufmann, 1989. 3
- [44] Aditya Prakash, Kashyap Chitta, and Andreas Geiger. Multi-modal fusion transformer for end-to-end autonomous driving. *CoRR*, abs/2104.09224, 2021. 3

- [45] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *ArXiv*, abs/2204.06125, 2022. 3
- [46] Katrin Renz, Kashyap Chitta, Otniel-Bogdan Mercea, A. Sophia Koepke, Zeynep Akata, and Andreas Geiger. Plant: Explainable planning transformers via object-level representations. In *6th Annual Conference on Robot Learning*, 2022. 2, 3, 6, 7
- [47] Moritz Reuss, Maximilian Li, Xiaogang Jia, and Rudolf Lioutikov. Goal-conditioned imitation learning using score-based diffusion policies. *arXiv preprint arXiv:2304.02532*, 2023. 3
- [48] Nicholas Rhinehart, Jeff He, Charles Packer, Matthew A Wright, Rowan McAllister, Joseph E Gonzalez, and Sergey Levine. Contingencies from observations: Tractable contingency planning with learned behavior models. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13663–13669. IEEE, 2021. 3, 8
- [49] Robin Rombach, A. Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10674–10685, 2022. 3
- [50] Reuven Y Rubinstein. Optimization of computer simulation models with rare events. *European Journal of Operational Research*, 99(1):89–112, 1997. 1, 7, 8
- [51] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L. Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, Seyedeh Sara Mahdavi, Raphael Gontijo Lopes, Tim Salimans, Jonathan Ho, David J. Fleet, and Mohammad Norouzi. Photorealistic text-to-image diffusion models with deep language understanding. *ArXiv*, abs/2205.11487, 2022. 3
- [52] Oliver Scheel, Luca Bergamini, Maciej Wolczyk, Błażej Osiński, and Peter Ondruska. Urban driver: Learning to drive from real-world demonstrations using policy gradients. In *Conference on Robot Learning*, 2021. 6
- [53] Oliver Scheel, Luca Bergamini, Maciej Wolczyk, Błażej Osiński, and Peter Ondruska. Urban driver: Learning to drive from real-world demonstrations using policy gradients. In *Conference on Robot Learning*, pages 718–728. PMLR, 2022. 2, 6, 7, 13
- [54] Ari Seff, Brian Cera, Dian Chen, Mason Ng, Aurick Zhou, Nigamaa Nayakanti, Khaled S Refaat, Rami Al-Rfou, and Benjamin Sapp. Motionlm: Multi-agent motion forecasting as language modeling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8579–8590, 2023. 13
- [55] Hao Sha, Yao Mu, Yuxuan Jiang, Li Chen, Chenfeng Xu, Ping Luo, Shengbo Eben Li, Masayoshi Tomizuka, Wei Zhan, and Mingyu Ding. Languagempc: Large language models as decision makers for autonomous driving. *arXiv preprint arXiv:2310.03026*, 2023. 3
- [56] Nur Muhammad Mahi Shafiullah, Zichen Jeff Cui, Ariuntuya Altanzaya, and Lerrel Pinto. Behavior transformers: Cloning k modes with one stone, 2022. 3
- [57] Jascha Narain Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. *ArXiv*, abs/1503.03585, 2015. 3
- [58] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020. 3, 4, 13
- [59] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021. 13
- [60] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning, 2023. 3
- [61] Yichuan Charlie Tang and Ruslan Salakhutdinov. Multiple futures prediction. *CoRR*, abs/1911.00997, 2019. 3
- [62] Julen Urain, Niklas Funk, Georgia Chalvatzaki, and Jan Peters. Se (3)-diffusionfields: Learning cost functions for joint grasp and motion optimization through diffusion. *ICRA*, 2023. 1
- [63] Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. *arXiv preprint arXiv:2208.06193*, 2022. 2
- [64] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H. Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *CoRR*, abs/2201.11903, 2022. 3
- [65] Licheng Wen, Xuemeng Yang, Daocheng Fu, Xiaofeng Wang, Pinlong Cai, Xin Li, Tao Ma, Yingxuan Li, Linran Xu, Dengke Shang, et al. On the road with gpt-4v (ision): Early explorations of visual-language model on autonomous driving. *arXiv preprint arXiv:2311.05332*, 2023. 3
- [66] Grady Williams, Andrew Aldrich, and Evangelos Theodorou. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149*, 2015. 1, 5, 8
- [67] Zhou Xian, Nikolaos Gkanatsios, Théophile Gervet, Tsung-Wei Ke, and Katerina Fragkiadaki. Chained-diffuser: Unifying trajectory diffusion and keypose prediction for robotic manipulation. In *CoRL 2023*. 3
- [68] Danfei Xu, Roberto Martín-Martín, De-An Huang, Yuke Zhu, Silvio Savarese, and Li Fei-Fei. Regres-

sion planning networks. *CoRR*, abs/1909.13072, 2019. 3

- [69] Zhenhua Xu, Yujia Zhang, Enze Xie, Zhen Zhao, Yong Guo, Kenneth KY Wong, Zhenguo Li, and Hengshuang Zhao. Drivegpt4: Interpretable end-to-end autonomous driving via large language model. *arXiv preprint arXiv:2310.01412*, 2023. 3
- [70] Mengjiao Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. *ArXiv*, abs/2310.06114, 2023. 3
- [71] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, et al. Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647*, 2023. 5
- [72] Wenyuan Zeng, Wenjie Luo, Simon Suo, Abbas Sadat, Bin Yang, Sergio Casas, and Raquel Urtasun. End-to-end interpretable neural motion planner. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8660–8669, 2019. 3
- [73] Zhejun Zhang, Alexander Liniger, Dengxin Dai, Fisher Yu, and Luc Van Gool. End-to-end urban driving by imitating a reinforcement learning coach. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021. 3
- [74] Ziyuan Zhong, Davis Rempe, Danfei Xu, Yuxiao Chen, Sushant Veer, Tong Che, Baishakhi Ray, and Marco Pavone. Guided conditional diffusion for controllable traffic simulation. *ArXiv*, abs/2210.17366, 2022. 3
- [75] Ziyuan Zhong, Davis Rempe, Danfei Xu, Yuxiao Chen, Sushant Veer, Tong Che, Baishakhi Ray, and Marco Pavone. Guided conditional diffusion for controllable traffic simulation. *ArXiv*, abs/2210.17366, 2022. 3
- [76] Yifeng Zhu, Jonathan Tremblay, Stan Birchfield, and Yuke Zhu. Hierarchical planning for long-horizon manipulation with geometric and symbolic scene graphs. *CoRR*, abs/2012.07277, 2020. 3
- [77] Julius Ziegler, Philipp Bender, Thao Dang, and Christoph Stiller. Trajectory planning for berth—a local, continuous method. In *2014 IEEE intelligent vehicles symposium proceedings*, pages 450–457. IEEE, 2014. 3

6. Appendix

6.1. Model training details

We train our diffusion models on 1% of the released nuPlan training data, which is subsampled from the original 20Hz to 0.5Hz. Our diffusion models are DDIMs [58] trained with $T = 100$ diffusion steps. We use the scaled linear beta schedule and predict ϵ . Our models are implemented in PyTorch and we use the HuggingFace Diffusers library to implement our diffusion models.

Our base diffusion architecture is as follows. Each trajectory waypoint is linearly projected to a latent feature with hidden size 256. The noise level is encoded with sinusoidal positional embeddings followed by a 2-layer MLP. Noise features are fused with the trajectory tokens by concatenating the noise feature to all trajectory features along the feature dimension and projecting back to the hidden size of 256. We also apply rotary positional embeddings [59] to the trajectory tokens as temporal embeddings. We then pass all the trajectory tokens through 8 transformer encoder layers, and each trajectory token is decoded to a corresponding waypoint. The final trajectory consists of the stacked waypoint predictions. Our conditional diffusion policy baseline uses a similar architecture, except we featurize the scene using the backbone from the nuPlan re-implementation of Urban Driver [53] and pass those tokens into the self-attention layers.

We train our models with batch size 256 and use the AdamW optimizer with learning rate $1e-4$, weight decay $5e-4$, and $(\beta_1, \beta_2) = (0.9, 0.999)$.

Our trajectories consist of 16 2D pose waypoints each with 3 features (x, y, θ) . We preprocess these trajectory features by applying Verlet wrapping as described in MotionLM [54], which we found to improve performance by encouraging smooth trajectories.

6.2. Language instruction following tasks

Here, we describe in detail each of the controllability tasks. For each, we list the task goal as well as the specific language instruction used.

1. **Lane change:** the ego-vehicle must execute a lane change. The language instruction is *"Change lanes to the left"*. The episode is considered a success if the ego-vehicle reaches the left lane.
2. **Unprotected left turn:** the ego-vehicle must perform an unprotected left turn. The language instruction is *"If car 18 is within 20 meters yield to it. Otherwise it will slow for you"*, where car 18 is the incoming car. The episode is considered a success if the ego-vehicle either completes the turn before the incoming car, or the incoming car passes the ego freely indicating a successful yield. Due to the randomized agent behaviors, it is not always possible to safely execute the turn in this task.
3. **Unprotected right turn:** the ego-vehicle must perform an unprotected right turn. The language instruction is *"Change to lane 33."*, where lane 33 is the target lane. The episode is considered a success if the ego-vehicle completes the right turn.
4. **Overtaking:** the ego-vehicle must overtake the target car. The language instruction is *"Car 21 will slow for you. Change to the right lane. Once ahead of car 4 change to the left lane."*, where car 21 is the incoming car in the right lane and car 4 is the target car. The episode is considered a success if the ego-vehicle is ahead of the target car while in the same lane.
5. **Extended overtaking:** the ego-vehicle must overtake the target car across several lanes of dense traffic. The language instruction is *"Change two lanes to the left. Then if you are ever ahead of car 3 change lanes to the right"*. The episode is considered a success if the ego-vehicle is ahead of the target car while in the same lane.
6. **Yielding:** the ego-vehicle must allow a car approaching quickly from behind to pass by changing lanes. The language instruction is *"Slow down and change lanes to the left. Then once car 8 is ahead of you change lanes to the right."*, where car 8 is the incoming car. The episode is considered a success if the ego-vehicle is behind the target car.
7. **Cut in:** the ego-vehicle must cut in to a column of cars. The language instruction is *"Slow down. Car 2 will slow for you. Change two lanes to the left"*, where car 2 is the car we are cutting in front of. The episode is considered a success if the ego-vehicle is in front of the target car and in the same lane.
8. **Lane weaving:** the ego-vehicle must reach a specific gap between two cars across several lanes of dense traffic. The language instruction is *"Once ahead of car 12 change lanes to the left. Then slow down a lot and change lanes to the left. Once car 9 is ahead of you by a few meters change lanes to the left"*. The episode is considered a success if the ego-vehicle successfully reaches the target gap.

6.3. Language instruction following prompts

We collect 24 instruction-program pairs in total, and they are shown in full below. To organize and manage our prompts, we use DSPy [30].

Listing 1. Full prompt with all instruction-program pairs

Use the provided language instruction to write code for guiding a lower-level driving policy.

Follow the following format.

Instruction: \${instruction}

Code: \${code}

Instruction: Yield to car 4. If car 2 is ahead of car 4 stop yielding.

Code:

```
done = self.yield_to_vehicle(self.get_vehicle(4))
def vehicle_ahead_of_other_vehicle():
    vehicle = self.get_vehicle(2)
    other_vehicle = self.get_vehicle(4)
    return vehicle.is_ahead_of(other_vehicle)
while not done() and not vehicle_ahead_of_other_vehicle():
    yield
self.stop_yielding()
```

Instruction: If car 1 is not stopped yield to it.

Code:

```
vehicle = self.get_vehicle(1)
if not vehicle.is_stopped():
    done = self.yield_to_vehicle(vehicle)
    while not done():
        yield
    self.stop_yielding()
```

Instruction: Change to the left lane.

Code:

```
done = self.follow_lane(self.left_lane)
while not done():
    yield
```

Instruction: If the speed of car 2 is lower than the speed limit change to the right lane.

Code:

```
vehicle = self.get_vehicle(2)
if vehicle.speed < self.current_lane.speed_limit:
    done = self.follow_lane(self.right_lane)
    while not done():
        yield
```

Instruction: Car 20 will slow down.

Code:

```
vehicle = self.get_vehicle(20)
self.adjust_constant_velocity_prediction(vehicle, 0.5)
```

Instruction: If car 17 is within 20 meters yield to it.

Code:

```
vehicle = self.get_vehicle(17)
if vehicle.distance_to(self.ego_vehicle) < 20.0:
    done = self.yield_to_vehicle(vehicle)
    while not done():
        yield
    self.stop_yielding()
```

Instruction: Resume normal driving speed.

Code: self.unset_ego_speed_limit()

Instruction: While you are ahead of car 3 stay in the current lane. Otherwise change to their lane.

Code:

```

def ahead_of_vehicle():
    vehicle = self.get_vehicle(3)
    return self.ego_vehicle.is_ahead_of(vehicle)
while ahead_of_vehicle():
    self.follow_lane(self.current_lane)
    yield
vehicle = self.get_vehicle(3)
their_lane = vehicle.get_closest_lane(self.lane_graph)
done = self.follow_lane(their_lane)
while not done():
    yield

---

Instruction: Follow lane 12.
Code:
lane = self.get_lane(12)
done = self.follow_lane(lane)
while not done():
    yield

---

Instruction: Once the speed of car 2 is lower than the speed limit of your current lane change to the right lane.
Code:
speed_limit = self.current_lane.speed_limit
def speed_exceeds_limit():
    vehicle = self.get_vehicle(2)
    return vehicle.speed < speed_limit
while speed_exceeds_limit():
    yield
done = self.follow_lane(self.right_lane)
while not done():
    yield

---

Instruction: Change to lane 1.
Code:
lane = self.get_lane(1)
done = self.follow_lane(lane)
while not done():
    yield

---

Instruction: Yield to car 3.
Code:
vehicle = self.get_vehicle(3)
done = self.yield_to_vehicle(vehicle)
while not done():
    yield
self.stop_yielding()

---

Instruction: If car 2 is ever ahead of you by 10 meters it will slow for you.
Code:
def vehicle_ahead_of_us():
    vehicle = self.get_vehicle(2)
    return vehicle.is_ahead_of(self.ego_vehicle, 10.0)
while not vehicle_ahead_of_us():
    yield
self.set_velocity_ratio(self.get_vehicle(2), 0.5)

---

Instruction: Change lanes to the right. Then change lanes to the left.
Code:
done = self.follow_lane(self.right_lane)
while not done():
    yield
done = self.follow_lane(self.left_lane)
while not done():
    yield

---

Instruction: If the speed of car 8 is greater than 5.0 yield to it.

```

```

Code:
vehicle = self.get_vehicle(8)
if vehicle.speed > 5.0:
    done = self.yield_to_vehicle(vehicle)
    while not done():
        yield
    self.stop_yielding()

---

Instruction: Change lanes to the right. Once car 1 is ahead of you yield to it.
Code:
done = self.follow_lane(self.right_lane)
while not done():
    yield
def vehicle_ahead_of_us():
    vehicle = self.get_vehicle(1)
    return vehicle.is_ahead_of(self.ego_vehicle)
while not vehicle_ahead_of_us():
    yield
done = self.yield_to_vehicle(self.get_vehicle(1))
while not done():
    yield
self.stop_yielding()

---

Instruction: Slow down and change lanes to the right.
Code:
current_speed = self.ego_vehicle.speed
self.set_ego_speed_limit(current_speed * 0.5)
done = self.follow_lane(self.right_lane)
while not done():
    yield

---

Instruction: Slow down a lot and change lanes to the right. Then once car 2 is ahead of you, resume normal driving
speed.
Code:
current_speed = self.ego_vehicle.speed
self.set_ego_speed_limit(current_speed * 0.2)
done = self.follow_lane(self.right_lane)
while not done():
    yield
def vehicle_ahead_of_us():
    vehicle = self.get_vehicle(2)
    return vehicle.is_ahead_of(self.ego_vehicle)
while not vehicle_ahead_of_us():
    yield
current_speed = self.ego_vehicle.speed
self.unset_ego_speed_limit()

---

Instruction: Stay in the current lane. If your speed exceeds 2.0 stop following the current lane.
Code:
done = self.follow_lane(self.current_lane)
def speed_under_threshold():
    speed = self.ego_vehicle.speed
    return speed < 2.0
while not done() and speed_under_threshold():
    yield
self.stop_following()

---

Instruction: While car 3 is ahead of you stay in the current lane. Otherwise change to their lane.
Code:
def vehicle_ahead_of_us():
    vehicle = self.get_vehicle(3)
    return vehicle.is_ahead_of(self.ego_vehicle)
while vehicle_ahead_of_us():
    self.follow_lane(self.current_lane)
    yield
vehicle = self.get_vehicle(3)
their_lane = vehicle.get_closest_lane(self.lane_graph)
done = self.follow_lane(their_lane)
while not done():

```

```

yield
---

Instruction: If there is a car in the left lane going faster than 5.0 m/s stay in the current lane.
Code:
vehicles_in_left_lane = self.left_lane.get_vehicles()
if any([vehicle.speed > 5.0 for vehicle in vehicles_in_left_lane]):
    done = self.follow_lane(self.current_lane)
    while not done():
        yield
---

Instruction: If car 2 is ahead of you by 10 meters it will slow for you.
Code:
vehicle = self.get_vehicle(2)
if vehicle.is_ahead_of(self.ego_vehicle, 10.0):
    self.adjust_constant_velocity_prediction(vehicle, 0.5)
---

Instruction: Stay in the current lane.
Code:
done = self.follow_lane(self.current_lane)
while not done():
    yield
---

Instruction: If car 20 is ever stopped yield to it.
Code:
def vehicle_is_ever_stopped():
    vehicle = self.get_vehicle(20)
    return vehicle.is_stopped()
while not vehicle_is_ever_stopped():
    yield
vehicle = self.get_vehicle(20)
done = self.yield_to_vehicle(vehicle)
while not done():
    yield
self.stop_yielding()
---

Instruction: [insert command here]
Code:

```